P3559R0 `is_trivially_relocatable`: One trait or two?

Published Proposal, 2025-01-07

Author:
Arthur O'Dwyer

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Trivial relocation is used by Abseil, AMC, BSL, Folly, HPX, Parlay, Pocketpy, Qt, Subspace, Thrust, and others, all with a single trait `is_trivially_relocatable` matching P1144's semantics. P2786 proposes two traits instead: P2786 changes `is_trivially_relocatable` to imply only a subset of its current meaning, and adds `is_replaceable` to fill in the gap. We argue that P2786's change will generate both inconvenience and actual bugs.

Table of Contents

References

Informative References

§ 1. Changelog

- R0 (pre-Hagenberg 2025): Initial revision.

§ 2. Background

"Relocation" is the quantity-preserving analog of "copying." A set of trivially copyable objects can have arbitrarily many *copies* of their values made at arbitrary memory locations simply by putting the right bytes there. A set of trivially relocatable objects can be *relocated* to arbitrary memory locations simply by putting the right bytes there if and only if the mapping from source values to destination values is 1:1.

Many important libraries care about this ability to arbitrarily relocate objects via memcpy: e.g. in `erase`, in `swap`, in the move-assignment operator of `inplace_vector`, etc. These optimizations are not always safe. These optimizations are safe *only* when the objects in question can have their values arbitrarily relocated in a way tantamount to memcpy. For example, it's not safe to optimize `swap(t, t)` into `__libcpp_memswap(&t, &t, __libcpp_datasizeof<T>)` unless `T` has this special property; it's not safe to optimize `inplace_vector<T>` move-assignment unless `T` has this special property; and so on. `tuple<int&>` lacks this property. `struct S {pmr::vector<int> v_;}` lacks this property. Both P1144 and P2786 agree on this baseline truth about when optimizations are safe to perform.

Let's call this property *P*. Many important libraries care about property *P*. They've written their own type-trait for *P*, and gate their optimizations on that type-trait.

The latest revision of [P2786R11] proposes two new traits for the Standard Library:

- `std::is_replaceable_v<T>` means roughly that assignment is tantamount to destroy-and-move-construct.
- `std::is_trivially_relocatable_v<T>` means that destroy-and-move-construct is tantamount to memcpy.

P2786 points out, correctly, that a library who cares about property *P* can write their own type-trait as:

```
template <class T>
constexpr bool has_property_p_v =
  std::is_trivially_relocatable_v<T> &&  // P2786 semantics
  std::is_replaceable_v<T>;
```

✧ ✧ ✧

[P1144], in contrast, has always proposed a single trait, named `std::is_trivially_relocatable_v<T>`, corresponding precisely to property *P*:

```
template <class T>
constexpr bool has_property_p_v =
  std::is_trivially_relocatable_v<T>;  // P1144 semantics
```

✧ ✧ ✧

What sorts of names do libraries actually pick, in practice, for this type-trait that I've been calling `has_property_p`?

Third-party library	*Name for property P*
Abseil	`absl::is_trivially_relocatable`
Amadeus AMC	`amc::is_trivially_relocatable`
Bloomberg BSL	`bslmf::IsBitwiseMoveable`
Folly	`folly::IsRelocatable`
HPX	`hpx::experimental::is_trivially_relocatable`
CMU Parlay	`parlay::is_trivially_relocatable`
PocketPy	`pkpy::is_trivially_relocatable_v`
Qt	`Q_IS_RELOCATABLE`
Google Skia	`sk_is_trivially_relocatable`
Subspace	`sus::mem::TriviallyRelocatable`
Thrust	`thrust::is_trivially_relocatable`
snncpp/snn-core	`snn::is_trivially_relocatable`
charles-salvia/std_error	`stdx::is_trivially_relocatable`
sarah-quinones/veg	`veg::trivially_relocatable`

Abseil, Folly, HPX, Parlay, and Subspace all explicitly define their trait in terms of P1144's `std::is_trivially_relocatable` whenever the compiler advertises support for P1144. A typical implementation goes like this:

```
namespace lib {
  // Define a trait for property P,
  // by the name is_trivially_relocatable.
  template <class T>
  struct is_trivially_relocatable :
  #if __cpp_lib_trivially_relocatable // P1144
    std::is_trivially_relocatable<T> {};
  #else
    std::is_trivially_copyable<T> {};
  #endif
}
~~~~
if constexpr (lib::is_trivially_relocatable<T>::value) {
  ~~~~ memcpy ~~~~
}
```

Notice that this relies on the Standard Library to define `std::is_trivially_relocatable` with P1144 semantics. That is, if their STL vendor decides to define the feature-test macro `__cpp_lib_trivially_relocatable` but to give `std::is_trivially_relocatable` some other semantics that are laxer than P1144's — for example, to give it P2786R11's proposed semantics instead —then this library will mis-optimize. For example, it will use memswap to swap objects that can't be swapped that way. It will use memmove to shift objects (e.g. in `vector::erase`) that can't be shifted that way.

P2786 basically asks that all third-party libraries should "fix themselves" by changing their code in one of two ways: Either they'll have to modify their definition of the property-*P* trait, like this:

```
namespace lib {
  // Define a trait for property P,
  // by the name is_trivially_relocatable.
  template <class T>
  struct is_trivially_relocatable :
  #if __cplusplus >= 20XXYYL  // P2786
    // Unfortunately the STL's version is weaker than ours
    std::bool_constant<std::is_trivially_relocatable_v<T> && std::is_replac
  #else
    std::is_trivially_copyable<T> {};
```

```
  #endif
}
~~~~
if constexpr (lib::is_trivially_relocatable<T>::value) {
  ~~~~ memcpy ~~~~
}
```

Or else they'll have let their type-trait's own meaning shift from property *P* to the weaker property *Q*, and carefully modify every call-site, like this:

```
namespace lib {
  // Define a trait for property Q (no longer P),
  // by the name is_trivially_relocatable.
  template <class T>
  struct is_trivially_relocatable :
  #if __cplusplus >= 20XXYYL  // P2786
    std::is_trivially_relocatable<T> {};
  #else
    std::is_trivially_move_constructible<T> {};
  #endif

  template <class T>
  struct is_replaceable :
  #if __cplusplus >= 20XXYYL  // P2786
    std::is_replaceable<T> {};
  #else
    std::is_trivially_copyable<T> {};
  #endif
}
~~~~
if constexpr (lib::is_trivially_relocatable<T>::value &&
               lib::is_replaceable<T>::value) {
  ~~~~ memcpy ~~~~
}
```

Neither of these are appealing solutions. Even worse, if neither solution is applied (or a solution is applied inconsistently), then the clients of `lib` will find themselves memcpying objects with property *Q* in situations where property *P* is required for safety — i.e., in situations where memcpy has the wrong physical behavior — leading to wild pointers, the eliding of important side-effects, or worse.

We should not put a trait named `std::is_trivially_relocatable` into the Standard Library unless it has the same *meaning* as today's library vendors understand by `lib::is_trivially_relocatable`. That is, property *P*.

§ 3. Libraries overwhelmingly prefer a single trait

In June 2024, Arthur catalogued 21 libraries that use traits or algorithms relating in some way to "trivial relocation."

13 of these libraries define a single trait for property *P* (that is, P1144 semantics). Of those, 8 of them explicitly refer to P1144 and implement the set of library algorithms and/or container optimizations that P1144 proposes. 5 libraries (listed in the timeline below) use P1144's feature-test macros.

Vice versa, 2 libraries define a single trait for property *Q* (that is, P2786 semantics): these are Thermadiag/seq and OleErikPeistorpet/OE-Lib. Their traits are true for `tuple<int&>`, although it's unclear if every use of these traits is correct.

As for standard libraries themselves: Both libc++ and libstdc++ define private traits which use P1144 semantics (they are false for `tuple<int&>`), but currently use their traits only to optimize vector reallocation, so they remain safe if either P1144 or P2786 is chosen. Microsoft STL does not perform any relocation optimizations.

No library ever defines anything corresponding to P2786's `is_replaceable` as a standalone trait.

No library defines both a trait for property *P* and one for property *Q*. That is, the 13 libraries that care about *P* do not seem to care about *Q*; and the 2 libraries that seem to care about *Q* do not seem to care about *P*. We can define a trait that captures most (but not all) of property *Q*...

```
namespace lib {
  template <class T>
  struct has_property_q :
    std::bool_constant<has_property_p<T> || std::is_trivially_move_construc
}
static_assert(lib::has_property_q<std::tuple<int&>>);
```

...but none of the 13 libraries above seem to care about capturing that particular set of semantics; they're happy to capture property *P* alone. *P* is the property most salient to their optimizations. We should give property *P* the good name.

§ 4. Timeline of P2786 and P1144

The committee history, as I understand it, goes like this:

2018	Oct	P1144 is published
2020	Feb	P1144 voted forward by EWGI (1–3–4–1–0); remains in EWGI
2023	Feb	P2786 is published
		P1144 voted forward by EWGI (0–7–4–3–1); remains in EWGI
		P2786 voted forward by EWGI (1–8–3–3–1); goes to EWG
2024	Feb	P2786 voted forward by EWG (7–9–6–0–2); goes to CWG
	Apr	[P3233] and [P3236] are published
	Jun	P2786 voted backward by EWG (21–15–3–6–5); goes back to EWG
	Nov	P2786 voted sideways by EWG (10-10-2-7-2); goes to LEWG
		P2786 voted sideways by LEWG (11-4-0-1-8); goes to EWG

The implementation history goes like this. For our purposes here, "P1144 semantics" means simply that a type with a funky assignment operator (such as `tuple<int&>`) *is not* trivially relocatable; "P2786 semantics" means simply that it *is*.

2012	Dec	Facebook Folly adds `IsRelocatable` with P1144 semantics
2013		Bloomberg BSL uses `BitwiseMoveable` with P1144 semantics
2015	Jul	Qt adds `isRelocatable` with P1144 semantics
	Sep	OleErikPeistorpet/OE-Lib adds `is_trivially_relocatable` with P1144 semantics
2018	Jul	Arthur O'Dwyer implements P1144 in a Clang fork, available on godbolt.org (full libc++ and libstdc++ support)
	Oct	P1144R0 is published
		libstdc++ adds `__is_bitwise_relocatable` with P1144 semantics
	Nov	OleErikPeistorpet/OE-Lib changes from P1144 to P2786 semantics
2021	Feb	sarah-quinones/veg adds `trivially_relocatable` with P1144 semantics
	Apr	Amadeus AMC uses `is_trivially_relocatable` with P1144 semantics
2022	May	Subspace adds `relocate_one_by_memcpy` with P2786 semantics
	Sep	snncpp/snn-core uses `is_trivially_relocatable` with P1144 semantics
	Oct	Google Skia adds `sk_is_trivially_relocatable` with P1144 semantics
2023	Jan	Subspace changes from P2786 to P1144 semantics
	Feb	P2786R0 is published
	Jul	Ste||ar HPX explores both P2786 and P1144 during GSoC 2023
	Oct	Quuxplusone/SG14 uses P1144's feature-test macro
	Nov	charles-salvia/std_error adds P1144's feature-test macro
2024	Jan	Ste||ar HPX adds `is_trivially_relocatable` with P1144 semantics

Feb	libc++ adds `__libcpp_is_trivially_relocatable` with compiler-dependent semantics CMU Parlay adds P1144's feature-test macro Pocketpy adds `is_trivially_relocatable_v` with P1144 semantics Google Abseil adds P1144's feature-test macro Corentin Jabot implements P2786 in a Clang fork, available on godbolt.org (no library support)
Mar	Amirreza Ashouri proposes Clang's `__is_trivially_relocatable` builtin should have P1144 semantics. Commenters in favor include AMC, HPX, Parlay, Qt, and Subspace
Apr	P3236R0 is published. Signatories include AMC, Blender, Folly, HPX, Parlay, and Qt
Jun	Facebook Folly adds P1144's feature-test macro
Aug	ultimatepp/ultimatepp adds `is_trivially_relocatable` with P1144 semantics

5. Learn more about trivial relocation

Besides Arthur's blog, you might also want to look at:

- "Trivially relocatable types in C++/Subspace", by Dana Jansens
- "Qt and trivial relocation", by Giuseppe D'Angelo, on the KDAB blog
- "Relocation semantics in the HPX library", by Isidoros Tsaousis
- "Object relocation" in the Folly repository

References

Informative References

[P1144]
Arthur O'Dwyer. *std::is_trivially_relocatable*. October 2024. URL: https://open-std.org/jtc1/sc22/wg21/docs/papers/2024/p1144r12.html

[P2786R11]
Mungo Gill; et al. *Trivial Relocatability For C++26*. December 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2786r11.html

[P3233]

Giuseppe D'Angelo. *Issues with P2786 (‘Trivial Relocatability For C++26’)*. April 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3233r0.html

[P3236]

Alan de Freitas; et al. *Please reject P2786 and accept P1144*. May 2024. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p3236r1.html